Autodesk®
Scaleform®

Scaleform

Scaleform 3.1 C++ Flash ActionScript

Mustafa Thamer Prasad Silva

2.01

2010 9 21

Autodesk®
GAMEWARE

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

Autodesk Scaleform

	Scaleform
	Autodesk Scaleform Corporation 6305 Ivy Lane, Suite 310 Greenbelt, MD 20770, USA www.scaleform.com
Email	info@scaleform.com
	+1 (301) 446-3200
Fax	+1 (301) 446-3199

目次

、 、 とのインタフェース

Flash® ActionScript™

ActionScript Flash
ActionScript

Flash
Autodesk Scaleform® ActionScript C++
Flash
ActionScript ActionScript
C++

C++ Flash

→	→
	ActionScript
	ActionScript
GFx::FunctionHandler ActionScript VM	GFx::Value

1.1 ActionScript から C++

Scaleform C++ ActionScript 2
FSCommand *ExternalInterface*
ActionScript API Flash FSCommand
ExternalInterface C++ Scaleform
Scaleform
GFx::Loader GFx::MovieDef GFx::Movie Scaleform
Scaleform

FSCommand ActionScript fscommand fscommand
ActionScript 2
2 const char C++
ActionScript fscommand C++
fscommand

ActionScript *flash.external.ExternalInterface.call* ExternalInterface
ActionScript
C++ ExternalInterface const char
ActionScript GFx::Value ActionScript
ExternalInterface.call C++ ExternalInterface
GFx::Value ActionScript ExternalInterface ActionScript Flash
Player Scaleform
External API

FSCommands ExternalInterface

FSCommands ExternalInterface Direct Access API GFx::FunctionHandler
C++
ActionScript VM
ActionScript C++
GFx::FunctionHandler Direct Access API

コールバック

ActionScript fscommand
ActionScript

```
fscommand("setMode", "2");
```

boolean integer fscommand

ActionScript fscommand GFx FSCommand 2
GFx::FSCommandHandler
fscommand GFx::Loader
GFx::MovieDef GFx::Movie
GFx::Loader -> GFx::MovieDef -> GFx::Movie

GFx::Movie GFx::RenderConfig
EdgeAA GFx::Translator
GFx::Loader
GFx::Movie
FSCommand
GFxPlayerTiny
FxPlayerFSCommandHandler

fscommand GFx::FSCommandHandler

```
class OurFSCommandHandler : public FSCommandHandler
{
    public:
    virtual void Callback(Movie* pmovie, const char* pcommand,
                          const char* parg)
    {
        Printf("FSCommand: %s, Args: %s", pcommand, parg);
    }
};
```

Callback ActionScript fscommand 2 fscommand

fscommand

GFx::Loader

```
// Register our fscommand handler
Ptr<FSCommandHandler> pcommandHandler = *new OurFSCommandHandler;
gfxLoader->SetFSCommandHandler(pcommandHandler);
```

GFx::Loader GFx::Movie GFx::MovieDef

SetFSCommandHandler

C++

Advance Invoke

外部インタフェース

Flash ExternalInterface.call fscommand

ExternalInterface

fscommand
ExternalInterface

```
class OurExternalInterfaceHandler : public ExternalInterface
{
    public:
    virtual void Callback(Movie* pmovieView, const char* methodName,
                                    const Value* args, unsigned argCount)
    {
        Printf("ExternalInterface: %s, %d args: ", methodName, argCount);
        for(unsigned i = 0; i < argCount; i++)
        {
              switch(args[i].GetType())
              {
                  case Value::VT_Number:
                    printf("%3.3f", args[i].GetNumber());
                    break;
                  case Value::VT_String:
                    printf("%s", args[i].GetString());
                    break;
              }
              printf("%s", (i == argCount - 1) ? "" : ", ");
        }
        printf("\n");

        // return a value of 100
        Value retValue;
        retValue.SetNumber(100);
        pmovieView->SetExternalInterfaceRetVal(retValue);
    }
};
```

GFx::Loader

```
Ptr<GFxExternalInterface> pEIHandler = *new OurExternalInterfaceHandler;
gfxLoader.SetExternalInterface(pEIHandler);
```

ActionScript ExternalInterface

ExternalInterface
ExternalInterface methodName
methodName
GFx::Value

FxDelegate
Apps\Samples\GameDelegate\FxGameDelegate.h

FxDelegate GFx::ExternalInterface FxDelegate
methodName
Register/Unregister Handlers

FxDelegate minimap FxDelegate HUD
FxDelegate minimap

```
// *** Minimap Begin
void FxPlayerApp::Accept(FxDelegateHandler::CallbackProcessor* cbreg)
{
    cbreg->Process("registerMiniMapView", FxPlayerApp::RegisterMiniMapView);

    cbreg->Process("enableSimulation", FxPlayerApp::EnableSimulation);
    cbreg->Process("changeMode", FxPlayerApp::ChangeMode);
    cbreg->Process("captureStats", FxPlayerApp::CaptureStats);

    cbreg->Process("loadSettings", FxPlayerApp::LoadSettings);
    cbreg->Process("numFriendliesChange", FxPlayerApp::NumFriendliesChange);
    cbreg->Process("numEnemiesChange", FxPlayerApp::NumEnemiesChange);
    cbreg->Process("numFlagsChange", FxPlayerApp::NumFlagsChange);
    cbreg->Process("numObjectivesChange", FxPlayerApp::NumObjectivesChange);
    cbreg->Process("numWaypointsChange", FxPlayerApp::NumWaypointsChange);
    cbreg->Process("playerMovementChange", FxPlayerApp::PlayerMovementChange);
```

```
    cbreg->Process("botMovementChange", FxPlayerApp::BotMovementChange);

    cbreg->Process("showingUI", FxPlayerApp::ShowingUI);
}
```

FxDelegate
FxDelegate

1.1.2.2 ActionScript から External Interface を使用する

ActionScript External Interface

```
    import flash.external.*;
    ExternalInterface.call("foo", arg);
```

foo arg 0 1
ExternalInterface API Flash

ExternalInterface ExternalInterface

```
    flash.external.ExternalInterface.call("foo", arg)
```

1.1.2.3 ExternalInterface.addCallback

ExternalInterface.addCallback ActionScript
C++
this
C++ GFx::Movie::Invoke()

AS コード :

```
    import flash.external.*;
    var txtField:TextField = this.createTextField("txtField",
                                    this.getNextHighestDepth(), 0, 0, 200, 50);
```

```
varaliasName:String = "setText";
var instance:Object = txtField;
var method:Function = SetText;
var wasSuccessful:Boolean = ExternalInterface.addCallback(aliasName, instance,
                                                          method);
function SetText()
{
    trace(this + ".SetText ");
    this.text = "INVOKED!";
}
```

txtField this SetText ActionScript

C++コード:

```
pMovie->Invoke("setText", "");
```

txtField.SetText INVOKED! txtField
GFx::Movie::Invoke() 1.2.2

1.2 C++から ActionScript

ActionScript C++
C++ Flash Scaleform
Scaleform ActionScript
ActionScript
C++

Direct Access API ActionScript C++
Direct Access API

変数の操作

Scaleform ActionScript GetVariable SetVariable
Direct Access API
2 Set/GetVariable Direct
Access API
Direct
Access GFx::Value::SetMember ActionScript
GetVariable GetVariable GFx::Value
Direct Access

GetVariable SetVariable ActionScript

```
int counter = (int)pHUDMovie ->GetVariableDouble("_root.counter");
counter++;
pHUDMovie->SetVariable("_root.counter", Value((double)counter));
```

GetVariableDouble _root.counter C++ Double
GetVariableDouble
SetVariable _root.counter 。GFx::Movie
GetVariable and SetVariable
。

SetVariable sticky GFx::Movie::SetVarType
3
Flash 3
_root.mytextfield
1 SetVariable(“_root.mytextfield.text”, “testing”,
SV_Normal) SV_Sticky
3

_root.mytextfield.text C++
SV_Normal SV_Sticky
SV_ Normal

Scaleform SetVariableArray and
GetVariableArray

SetVariableArray

GFx::Movie::SetVariableArraySize

SetVariableArray AS

```
int idxInASArray = 0;
const char* strarr[2];
strarr[0] = "This is the first string";
strarr[1] = "This is the second one";
pMovie->SetVariableArray(Movie::SA_String, "_root.strArray",
                         idxInASArray, strarr, 2);
```

```
int idxInASArray = 0;
const wchar_t* strarr[2];
strarr[0] = "This is the first string";
strarr[1] = "This is the second one";
pMovie->SetVariableArray(Movie::SA_StringW, "_root.strArray",
                         idxInASArray, strarr, 2)
```

GetVariableArray AS

サブルーチンの実行

ActionScript 、GFx::Movie::Invoke() ActionScript

UI UI

Direct Access API

ActionScript

_root mySlider SetSliderPos mySlider

コード：

```
this.SetSliderPos = function (pos)
{
   // Clamp the incoming position value
   if (pos<rangeMin) pos = rangeMin;
   if (pos>rangeMax) pos = rangeMax;
   gripClip._x = trackClip._width * ((pos-rangeMin)/(rangeMax-rangeMin));
   gripClip.gripPos = gripClip._x;
}
```

gripClip mySlider pos rangeMin/Max

Invoke

コード：

```
Value result;
bool bInvoked = pMovie->Invoke("_root.mySlider.SetSliderPos", &result, "%d",
                               newPos);
```

Invoke true false

ActionScript ActionScript
Invoke ActionScript
Scaleform ActionScript

1
ActionScript GFx::Movie::Advance initFirstFrame
true GFx::MovieDef::CreateInstance

Invoke GFx::Movie::IsAvailable() AS

```
Value result;
if (pMovie->IsAvailable("parentPath.mySlider.SetSliderPos"))
   pMovie->Invoke("parentPath.mySlider.SetSliderPos", &result, "%d", newPos);
```

Invoke printf Invoke

GFx::Value

```
Value args[3], result;
args[0].SetNumber(i);
args[1].SetString("test");
args[2].SetNumber(3.5);
pMovie->Invoke("path.to.methodName", &result, args, 3);
```

InvokeArgs va_list
Invoke Invoke InvokeArgs printf vprintf

パス

Flash

GFx::Movie

```
Movie::IsAvailable(path)
Movie::SetVariable(path, value)
Movie::SetVariableDouble(path, value)
Movie::SetVariableArray(path, index, data, count)
Movie::SetVariableArraySize(path, count)
Movie::GetVariable(value, path)
Movie::GetVariableDouble( path)
Movie::GetVariableArray(path, index, data, count)
Movie::GetVariableArraySize(path)
Movie::Invoke(pathToMethodName, result, argList, ...)
```

.

clip2 clip1

Main Stage -> clip1 -> clip2
clip1
clip2 clip1

```
"clip1.clip2"
"_root.clip1.clip2"
"_level0.clip1.clip2"
```

```
"Clip1.clip2"  <-                    Clip1
"clip2"        <-      Clip1
```

"_root" "_level0" ActionScript 2
ActionScript 3
ActionScript 3
_root _level0 root level0
SWF _root
_levelN

Flash Studio Test Movie Ctrl-Enter Ctrl-Alt-V

Scene 1 Flash Studio

Ctrl-Alt-V

loadMovie
_level0 _level1
_level2 Level MovieClip
Object _levelN.objectName
_levelN.objectPath.objectName N

1. Paths to Objects and Variables Jesse Stratford
2. Advanced Pathing Jesse Stratford

Scaleform 3.1 GFx::Value Direct Access AS
Action Script
API

UI
Direct Access API HUD Kit minimap
Flash

Direct Access API GFx::Values ActionScript Object Array
Display Object Display Objects
Object MovieClip Button TextFields
GFx::Value API
GFx::Movie API
GFx::Value

Direct Access API ActionScript GFx::Value
C++
ActionScript
GFx::Movie AS
AS

GFx::Movie GFx::Value
ActionScript AS
foo
Movie Invoke
Invoke

1. GFx::Movie Invoke

```
Value ret;
Value args[N];
pMovie->Invoke("_root.foo.method", &ret, args, N);
```

2. Direct Access Invoke

```
(Assumes 'foo' holds a reference to an AS object at "_root.foo")
Value ret;
Value args[N];
bool = foo.Invoke("method", &ret, args, N);
```

Invoke AS
GFx::Value::HasMember GetMember SetMember Direct Access API

movieClip GFx::Value GetMember

```
Value tf;
movieClip.GetMember("textField", &tf);
tf.SetText("hello");
```

movieClip textfield SetText

2.1 オブジェクトとアレイ

Direct Access API AS

C++ snippet
2

```
Movie* pMovie = … ;
Value parent, child;
pMovie->CreateObject(&parent);
pMovie->CreateObject(&child);
parent.SetMember(“child”, child);
```

CreateObject ActionScript
CreateObject

```
Value mat, params[6];
// (set params[0..6] to matrix data) …
pMovie->CreateObject(&mat, "flash.geom.Matrix", params, 6);
```

Object Array

```
// (Assuming pMovie is a GFx::Movie*):
Value owner, childArr, childObj;
pMovie->CreateArray(&owner);          // create parent array
pMovie->CreateArray(&childArr);       // create child array
pMovie->CreateObject(&childObj);      // create child object
bool = owner.SetElement(0, childArr);  //set parent[0] to child array
bool = owner.SetElement(1, childObj); // set parent[1]to child object
```

```
…
bool = foo.SetMember("owner", owner); // later, set the 'owner' variable in
                                      // object foo, to the parent object
```

関数 GFx::Value::VisitMembers() Visit ObjectVisitor Array DisplayObject *VisitMembers* を使うことはできませんので注意してください。 enumerable ActionScript ASSetPropFlags

例: `_global.ASSetPropFlags(MyClass.prototype, ["someFunc", "__get__number", "test"], 6, 1);`

getter/setter ASSetPropFlags VisitMembers enumerable Direct Access VT_Object VT_Object ASSetPropFlags enumerable

2.2 ディスプレイオブジェクト

Direct Access API DisplayObject Object MovieClip Button TextField DisplayInfo API DisplayInfo API DisplayObject SetMember SetDisplayInfo GFx::Movie::SetVariable

SetDisplayInfo movieclip

```
// Assumes movieClip is a GFx::Value*
Value::DisplayInfo info;
PointF pt(100,100);
info.SetRotation(90);
info.SetPosition(pt.x, pt.y);
movieClip.SetDisplayInfo(info);
```

2.3 関数オブジェクト

ActionScript2 Virtual Machine AS2 VM

GFx::Movie::CreateFunction C++
CreateFunction
VM AS
C++ VM

fscommand ExternalInterface

AS

```
Value obj;
pmovie->GetVariable(&obj, "_root.obj");

class MyFunc : public FunctionHandler
{
    public:
        virtual void Call(const Params& params)
        {
              // Callback logic/handling
        }
};
Ptr<MyFunc> customFunc = *SF_HEAP_NEW(Memory::GetGlobalHeap()) MyFunc();
Value func;
pmovie->CreateFunction(&func, customFunc);
obj.SetMember("func", func);
```

ActionScript _root timeline

```
obj.func(param1, param2, param3);
```

Call() Param

- pRetVal: GFx::Value
  pMovie->CreateObject() CreateArray()
  Number Boolean

- pMovie:
- pThis:

- ArgCount:
- pArgs: GFx::Values [ ]

```
Value firstArg = params.pArgs[0];
```

- pArgsWithThisRef:

- pUserData:

VM

VM

```
Value networkProto, networkCtorFn, networkConnectFn;
class NetworkClass : public FunctionHandler
{
  public:
    enum Method
    {
        METHOD_Ctor,
        METHOD_Connet,
    };

    virtual ~NetworkClass() {}
    virtual void Call(const Params& params)
    {
        int method = int(params.pUserData);
        switch (method)
        {
            case METHOD_Ctor:
            // Custom logic
            break;
            case METHOD_Connet:
            // Custom logic
            break;
        }
    }
};
Ptr<NetworkClass> networkClassDef = *SF_HEAP_NEW(Memory::GetGlobalHeap())
                                                    NetworkClass();
// Create the constructor function
pmovie->CreateFunction(&networkCtorFn, networkClassDef,
                       (void*)NetworkClass::METHOD_Ctor);
// Create the prototype object
pmovie->CreateObject(&networkProto);
// Set the prototype on the constructor function
networkCtorFn.SetMember("prototype", networkProto);
```

```
        // Create the prototype method for 'connect'
        pmovie->CreateFunction(&networkConnectFn, networkClassDef,
                               (void*)NetworkClass::METHOD_Connet);
        networkProto.SetMember("connect", networkConnectFn);
        // Register the constructor function with _global
        pmovie->SetVariable("_global.Network", networkCtorFn);
```

VM

```
        var netObj:Object = new Network(param1, param2);
```

VM VM

```
        Value origFuncReal;
        obj.GetMember("funcReal", &origFuncReal);
        class FuncRealIntercept : public FunctionHandler
        {
            Value OrigFunc;
            public:
                FuncRealIntercept(Value origFunc) : OrigFunc(origFunc) {}
                virtual ~FuncRealIntercept() {}
                virtual void Call(const Params& params)
                {
                      // Intercept logic (beginning)
                      OrigFunc.Invoke("call", params.pRetVal, params.pArgsWithThisRef,
                                      params.ArgCount + 1);
                      // Intercept logic (end)
                }
        };
        Value funcRealIntercept;
        Ptr<FuncRealIntercept> ;};

    > };

l", , 1s Tj /C,;1s Tj34BDC8EMC9.MC Tm
```

オブジェクトのサポート

説明	パブリックメソッド "_root.foo" 'foo'
	`bool has = foo.HasMember("bar");`
	`Value bar;` `bool = foo.GetMember("bar", &bar);`
	`Value val;` `bool = foo.SetMember("bar", val);`

	`Value ret;`
	`Value args[N];`
	`bool = foo.Invoke("method", args, N, &ret);` *or*
	`foo.Invoke("method", args, N);`

	`Value obj;`
	`pMovie->CreateObject(&obj);`
	…
	`bool = foo.SetMember("bar", obj);`

	`Value owner, child;`
	`pMovie->CreateObject(&owner);`
	`pMovie->CreateObject(&child);`
	`bool = owner.SetMember("child", child);`
	…
	`bool = foo.SetMember("owner", owner);`

	`Value::ObjectVisitor v;` `foo.VisitMembers(&v);`
	`bool = foo.DeleteMember("bar");`

アレイのサポート

説明	パブリックメソッド "_root.foo.bar" 'bar' 'foo'
	`unsigned sz = bar.GetArraySize();`

	Value val; bool = bar.GetElement(idx, &val);
	Value val; bool = bar.SetElement(idx, val);
	bool = bar.SetArraySize(N);

	Value arr;
	pMovie->CreateArray(&arr);
	…
	bool = foo.SetMember("bar", arr);

	Value owner, childArr, childObj;
	pMovie->CreateArray(&owner);
	pMovie->CreateArray(&childArr);
	pMovie->CreateObject(&childObj);
	bool = owner.SetElement(0, childArr);
	bool = owner.SetElement(1, childObj);
	…
	bool = foo.SetMember("owner", owner);

	Enumeration
	for (UPInt i=0; i <N; i++)
	{ … bool = bar.GetElement(i+idx, &val) … }
	
	Using a visitor pattern:
	Value::ArrayVisitor v;
	bar.VisitElements(v, idx, N);

	bool = bar.ClearElements();

	PushBack
	Value val;
	bool = bar.PushBack(val);
	
	PopBack
	Value val;
	bool = bar.PopBack(&val); *or* void bar.PopBack();

	Single element
	bool = bar.RemoveElement(idx);
	
	Series of elements
	bool = bar.RemoveElements(idx, N);

ディスプレイオブジェクトのサポート

説明	**パブリックメソッド** "_root.foo.bar" MovieClip TextField Button 'foo'

	Value::DisplayInfo info; bool = foo.GetDisplayInfo(&info);
	Value::DisplayInfo info; … bool = foo.SetDisplayInfo(info);
	Matrix2_3 mat; bool = foo.GetDisplayMatrix(&mat);
	Matrix2_3 mat; … bool = foo.SetDisplayMatrix(mat);
	Render::Cxform cxform; bool = foo.GetColorTransform(&cxform);
	Render::Cxform cxform; … bool = foo.SetColorTransform(cxform);

のサポート

説明	パブリックメソッド "_root.foo" MovieClip 'foo'
MovieClip	Value newInstance; bool = foo.AttachMovie(&newInstance, "SymbolName", "instanceName");
MovieClip MovieClip	Value emptyInstance; bool = foo.CreateEmptyMovieClip(&emptyInstance, "instanceName");
keyframe	bool = foo.GotoAndPlay("myframe");
keyframe	bool = foo.GotoAndPlay(3);
keyframe	bool = foo.GotoAndStop("myframe");
keyframe	bool = foo.GotoAndStop(3);

のサポート

説明	パブリックメソッド "_root.foo" Textfield 'foo'
raw textField	Value text; bool = foo.GetText(&text);
raw textField	Value text; … bool = bar.SetText(text);

HTML	Value htmlText; bool = bar.GetTextHTML (&htmlText);
HTML	Value htmlText; … bool = bar.SetTextHTML(htmlText);